

Fit for Purpose: Selecting a Reification Method for the TIDO Ontology

Ritten Roothaert^{1,*}, Lise Stork² and María Poveda-Villalón³

¹Vrije Universiteit Amsterdam, De Boelelaan 1105, 1081 HV Amsterdam, The Netherlands

²University of Amsterdam, Science Park 900, 1098 XH Amsterdam, The Netherlands

³Universidad Politécnica de Madrid, Campus de Montegancedo, 28660 Boadilla del Monte, Madrid, Spain

Abstract

Knowledge graphs built from ontologies such as the Threat Intelligence Decision Ontology (TIDO) often need to express meta-information about individual statements, for example regarding provenance, confidence, or the perspective of the analyst asserting them. While RDF supports this to some extent through standard reification, this method comes with its own limitations, and several alternative reification methods have since been proposed, each with distinct trade-offs in semantics, expressivity, and performance. This paper presents a set of seven requirements for a reification method suited to the threat intelligence (TI) domain, grounded in a motivating example based on the TIDO ontology. Five commonly used reification methods, standard reification, n-ary relations, singleton properties, named graphs, and RDF-star, are compared against these requirements, combining an analysis of their inherent semantic properties with a review of previously published empirical comparisons of storage requirements, query execution performance, and user-friendliness. The presented analysis shows that named graphs satisfy the majority of identified requirements without deviating from the standard implementation, making it the most suitable reification method for TIDO-based knowledge graphs. Standard reification and n-ary relations are also viable options but would require non-standard implementations.

Keywords

RDF reification, Threat intelligence, Requirement analysis

1. Introduction

Threat intelligence (TI) analysts are tasked with making sense of large volumes of fragmented, uncertain, and often contradictory information in order to support decisions with significant consequences, ranging from the deployment of exceptional investigative means to the drafting of formal threat assessments. Unlike many other data-intensive domains, the value of a TI knowledge graph lies not only in the facts it records, but in the process by which those facts were interpreted, weighed, and turned into judgements. Decisions made by an analyst are frequently based on incomplete evidence, subjective interpretation, and evolving understanding, and the reasoning behind these decisions is rarely made explicit or preserved in a structured, machine-interpretable way. This creates a gap: while the outcomes of TI investigations may be recorded, the underlying decision processes, and the evidentiary reasoning that produced them, typically are not. For accountability, auditing, and knowledge-sharing purposes, however, it is precisely this decision process that needs to be documented.

To address this gap, the TIDO ontology (Threat Intelligence Decision Ontology) was developed to provide a unified and machine-interpretable vocabulary to represent the work processes of TI analysts, including the evidence they consider, the hypotheses they form, and the relations between them [1]. However, several statements that analysts can express using TIDO, such as which evidence supports or disputes a hypothesis, are not universally true facts but reflect the perspective of a specific analyst at a specific point in time, and need to be complemented with additional meta-information regarding,

WOP at ISWC'26: 17th Workshop on Ontology Design and Patterns at the International Semantic Web Conference, 25-29 October, 2026, Bari, Italy

*Corresponding author.

✉ h.m.roothaert@vu.nl (R. Roothaert); s.stork@uva.nl (L. Stork); mpoveda@fi.upm.es (M. Poveda-Villalón)

🌐 <https://lisestork.github.io/> (L. Stork); <https://mariapoveda.github.io/> (M. Poveda-Villalón)

🆔 0009-0008-7843-6513 (R. Roothaert); 0000-0002-2146-4803 (L. Stork); 0000-0003-3587-0367 (M. Poveda-Villalón)



© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

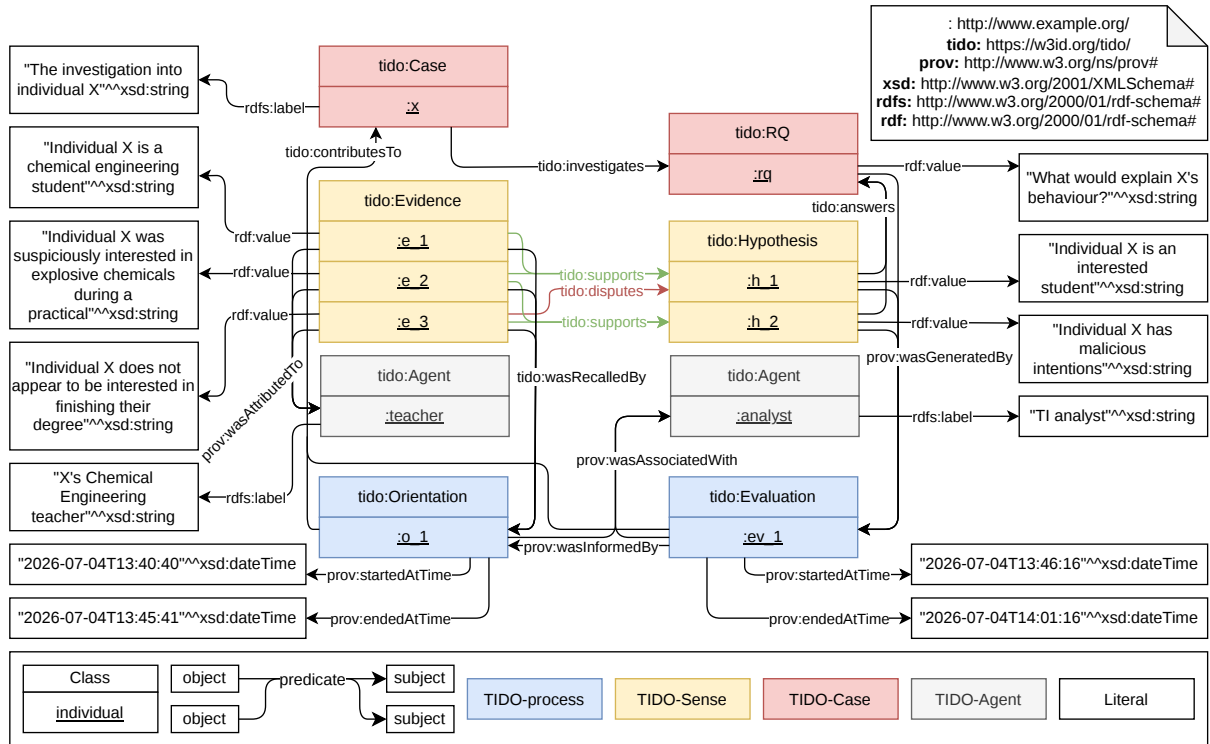


Figure 1: Minimal example of the representation of a processes of a TI analyst analysing the situation, according to the TIDO vocabulary. Note: for the sake of readability, `prov:wasMemberOf` relations were omitted.

e.g., confidence, provenance or access restrictions. Expressing such meta-information requires making statements about statements, and although RDF natively supports this through standard reification, this method comes with well-known limitations, motivating a range of alternative approaches, including n-ary relations, singleton properties, named graphs, and RDF-star.

Selecting an appropriate reification method is non-trivial, as each comes with different trade-offs in semantics, expressivity, and performance. This has motivated several prior comparisons, e.g. with respect to storage and query performance [2, 3], the representation of multi-valued properties [4], or suitability for specific applications [5]. However, these comparisons are scattered across different evaluation setups and none specifically considers the requirements of the TI domain.

This paper addresses this gap by combining insights from the existing comparative literature with a requirements analysis grounded in the specific needs of the TIDO ontology, in order to identify the reification method best suited for representing meta-information within a TIDO-based knowledge graph. Rather than conducting a new empirical benchmark, qualitative and quantitative findings are synthesised from prior comparisons, and each reification method is evaluated against requirements derived from the practical challenges a system storing TI analysts' decisions might face.

The remainder of this paper is structured as follows. Section 2 introduces a motivating example based on the TIDO ontology and uses it to derive a set of seven requirements that a reification method should satisfy in order to support TI decision processes. Section 3 provides a description of the five reification methods considered in this comparison. Section 4 evaluates these methods against the identified requirements, combining an analysis of their inherent properties with a literature review of previously published quantitative comparisons. Section 5 discusses the limitations of this analysis and outlines directions for future work, and Section 6 concludes the paper.

2. Motivating Example

To showcase why reification of the certain statements within the TIDO ontology is required, a minimal example is provided in Figure 1. In this example, a threat intelligence analyst (:analyst) is assigned the case of individual X (:x), which revolves around information provided by a chemical engineering teacher (:teacher). During an initial orientation step (:o_1), the analyst recalls three pieces of evidence provided by :teacher: the report is about one of his students (:e_1), this student was suspiciously interested in explosive chemicals during one of the practicals (:e_2), and the teacher claims that this student does not appear to be interested in finishing their degree programme (:e_3).

During a follow-up evaluation step (:ev_1), an :analyst asks an initial research question (:rq): “What would explain X’s behaviour?”. Additionally, the :analyst drafts two hypotheses that would answer :rq: The student is simply an interested student (:h_1), or the student has actual malicious intentions (:h_2). The available evidence is related to these hypotheses through the tido:supports or tido:disputes relations, where :e_1 supports :h_1, :e_2 supports both :h_1 and :h_2, and :e_3 supports :h_2 and disputes :h_1.

Challenge one: Multi-perspective validity. The threat intelligence domain, similar to most large-scale knowledge graphs, can be messy. Information is often missing or incomplete, and context can be as important as the information itself. For example, the statement “Individual X does not appear to be interested in finishing their degree” is nothing out of the ordinary under most conditions, but if the same individual X is still very interested in explosive chemicals, this *could be* an indication of malicious intent, though an analyst could equally interpret it (implicitly) as common adolescent behaviour. Relating this to Figure 1, the decision to include the triple $\langle :e_3 \text{ tido:supports } :h_2 \rangle$ is therefore equally valid as the decision to leave it out. Instances of tido:supports, tido:disputes, and tido:answers should thus be interpreted not as universal truths, but as the perspective of an actor, and multiple such perspectives should be able to co-exist within a TIDO-based KG.

Requirement 1

It MUST be possible for multiple perspectives to co-exists within the same KG

Challenge two: Dynamic nature of information. Information within the TI domain is not a static entity. New information may surface, or existing information may turn out to have been misinterpreted, for example when a source later proves unreliable, changing the context in which already known information should be interpreted. This forces an existing KG to be updated. However, between the initial assertion and the update, critical decisions could have been made, so for auditing reasons it must be possible to reconstruct the situational understanding at decision time, and therefore the old assertions should be preserved.

Requirement 2

It MUST be possible reconstruct past states

Challenge three: Merging duplicate statement reifications. In a multi-agent environment, an identical statement could be asserted on multiple occasions, each with its own reification. When independently generated graphs are merged, some reifications should be merged into one: if the reification of $\langle :e_3 \text{ prov:wasAttributedTo } :teacher \rangle$ occurs in both G_1 and G_2 , it should occur only *once* in $G_1 \cup G_2$. Others, however, should remain distinct: if the reification of $\langle :e_3 \text{ tido:supports } :h_2 \rangle$ occurs in both graphs, it should occur *twice* in $G_1 \cup G_2$, each accompanied by a different set of meta-data. Hence, it must be possible to define the merging behaviour of reified statements.

Requirement 3

It MUST be possible define reified statement merging behaviour at statement level.

Challenge four: Access restrictions. TI analysts have access to classified information. The information an analyst can access depends on the role within the institute, the clearances they have, and whether they have a justification for accessing this information. It should be possible to manage these access restrictions at the statement level. Note that this also includes the scenario where an analyst has access to a reified statement, but not to the original.

Requirement 4

It **MUST** be possible set access restrictions at statement level.

Challenge five: User-friendliness. TI analysts are highly educated, but have varying backgrounds. These backgrounds are not necessarily technical in nature and, therefore, they might not be familiar with querying databases. To minimise training requirements, the complexity of queries should be minimised.

Requirement 5

The representation **SHOULD** be optimised for minimal query creation complexity.

Challenge six: Optimisation. Providing an estimate of the size of an institute-wide KG is difficult, as operational information of intelligence services is under most circumstances classified. However, the 2026 annual statistical transparency report of the U.S. intelligence community in an exception and can be used to get a rough estimate of the scale of an institute-wide KG. It reports that almost 350,000 non-U.S. individuals were identified users of ‘tasking selectors’ (e.g., telephone number or email address) [6]. These individuals meet three criteria: (1) they are non-U.S. citizens, (2) they are reasonably believed to be located outside of the United States, and (3) they must be expected to possess, receive, and/or be likely to communicate foreign intelligence information. If the TIDO ontology were used to document decisions establishing that each of these criteria is satisfied, the resulting graph would easily exceed multiple millions of statements, and this excludes investigations into U.S. individuals, threat assessments, or decisions to deploy exceptional means. As this could be considered a relatively large KG, the effect of the reification method on the storage requirements and query execution times should also be considered.

Requirement 6

The representation **SHOULD** be optimised for minimising storage requirements

Requirement 7

The representation **SHOULD** be optimised for minimising query execution time.

3. Reification Methods

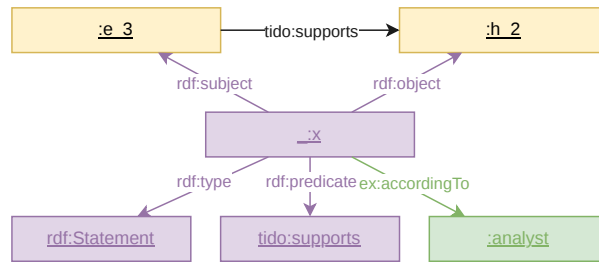
Scope definition. The objective of this analysis is to find a single reification method capable of satisfying (a majority of) the identified requirements. Reification methods are not mutually exclusive and can be used together, but using different methods simultaneously would increase the complexity of the ontology and reduce its interpretability. Hence, methods that focus only on temporal, confidence, or provenance information were excluded from the analysis. Only reification methods that make no assumptions about the type of information were included in the analysis. In the literature, the following five reification methods occurred most often: standard reification, N-ary relations, singleton properties, named graphs, and RDF-star. Both companion properties [7] and k-partite graphs are also capable of expressing any form of information [8], but adoption of these methods is substantially less compared to the other methods. The following section provides a brief description of the included methods, as well as differentiating characteristics with respect to semantics, inferences, and statement merging behaviour.

Listing 1: s-reif reified triple

```

1 :e_3 tido:supports :h_2 .
2 _:x rdf:type rdf:Statement ;
3   rdf:subject :e_3 ;
4   rdf:predicate tido:supports ;
5   rdf:object :h_2 ;
6   :accordingTo :analyst .

```

Figure 2: s-reif reified graph

Terminology. Throughout this section, the term *statement* is used to refer to the text used to describe a fact. In standard RDF, a statement corresponds to a triple, whereas in named graphs it corresponds to a quad. The term *reified statement* is used to refer to the identifier used to refer to a statement. For standard reification and n-ary relations, the reified statement is the introduced (blank) node; for singleton properties, it is the asserted singleton property; for named graphs, it is the graph to which the statement belongs; and for RDF-star, it is the quoted triple. Finally, the term *original statement* is used to refer to the statement to which a reified statement refers.

RDF reification (s-reif). Standard reification was introduced during the original publication of RDF as a W3C recommendation as a means to express statements about RDF statements [9]. They describe how the standard RDF vocabulary, `rdf:object`, `rdf:predicate`, `rdf:subject`, and `rdf:Statement`, can be used to create a (blank) node representing the original statement. Meta-data about this triple can be expressed by referencing this (blank) node. Note that the reified triple does not entail the original triple. An RDF graph only contains a fact given in the triple if and only if the triple is present in the graph, irrespective of whether the corresponding reified triple is present. The same interpretation is described in RDF 1.0 [10] and RDF 1.1 [11], although the latter specifies that the section on reification is no longer normative. However, many standard reification implementations do not assert the original triple and only focus on connecting meta-information to the reified statement [2, 7, 3, 4, 5, 12].

N-ary relations (n-rel). Where standard reification is intended to be used to make statements about statements, n-ary relations are intended to be used to describe relationships between more than two entities [13]. This idea is also used to express meta-information about statements. Instead of reifying the entire statement into a node, n-ary relationships adjust the vocabulary: The relationship expressed within the statement can be expressed as either an IRI or a blank node. This IRI or blank node is connected to the subject and object of the original statement by two introduced predicates. Whether the original statement should be inferred from this n-ary relation is considered a design decision. The PROV ontology uses n-ary relations to create ‘qualified’ versions of standard relations, for example, to specify to which agent a statement should be attributed. The PROV-O specifications state that unqualified statements should be inferred from a qualified statement, but recommend explicitly expressing both [14]. This is a deviation from Wiki-data’s original proposal for serialising their graph into an RDF dataset. They provide two distinct serialisations: One simple version without reified statements and one full version with only reified statements [15]. Although they consider the simple version not to be truthful, they acknowledge that it is still meaningful and easier to query. The example given in Listing 2 and Figure 3 is inspired by Wiki-data’s original proposal to serialise their graph.

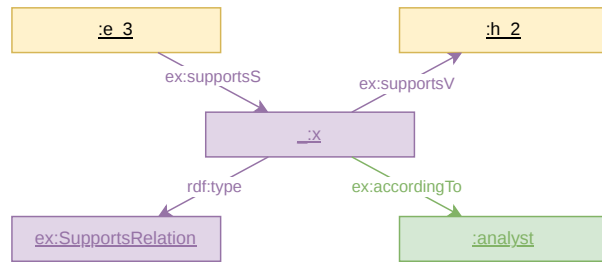
Singleton properties (s-prop). Singleton properties were introduced to reduce the number of statements needed to reify a statement [16]. Here, statements are reified by assigning a unique identifier to the predicate/property used within the statement creating a singleton instance of this property. The authors provide two interpretations of this singleton property: (1) the RDF interpretation and (2) the RDFS interpretation. For the RDF interpretation, a singleton property is interpreted as an instance of the generic property. Under this interpretation, a statement using a singleton property

Listing 2: n-rel reified triple

```

1 :e_3 ex:supportsS _:x .
2 _:x rdf:type ex:SupportsRelation ;
3   ex:supportsV :h_3 ;
4   ex:accordingTo :analyst .

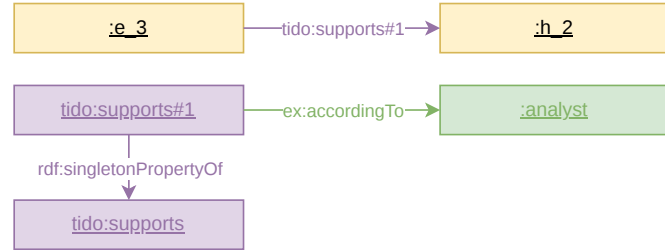
```

Figure 3: n-rel reified graph**Listing 3: s-prop reified triple**

```

1 :e_3 :supports#1 :h_2 .
2 tido:supports#1 rdf:singletonPropertyOf
3   tido:supports ;
4   ex:accordingTo :analyst .

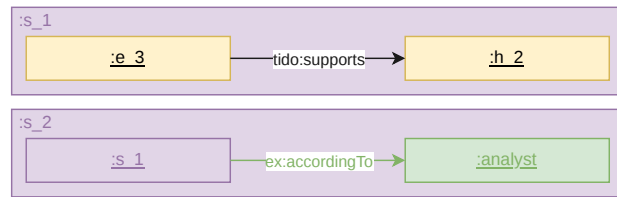
```

Figure 4: s-prop reified graph**Listing 4: n-graph reified triple**

```

1 :s_1 {
2   :e_3 tido:supports :h_2 . }
3 :s_2 {
4   :s_1 ex:accordingTo :analyst . }

```

Figure 5: n-graph reified graph.

does not entail a statement using the generic property. For a statement with the generic property to be considered a fact, it needs to be asserted separately. For the RDFS interpretation, a singleton property is considered to be a sub-property of the generic property. In fact, the model-theoretic semantics closely resemble the semantics of the `rdfs:subPropertyOf` relation [10], with the addition of enforcing the uniqueness of the singleton properties. Under this interpretation, a statement using a singleton property *does* entail a statement using the generic property. In the comparisons found, the distinction between the two interpretations is often not explicitly mentioned, but the RDFS interpretation is more common [17, 7] and will therefore be used in this manuscript.

Named graphs (n-graph). Named graphs can also be used to refer to individual statements [18]. This method for reifying triples was introduced to provide greater precision and potential interoperability with standard reification and other ad-hoc RDF extensions available at the time of publishing. Additionally, when combined with the Semantic Web Publishing vocabulary [19], named graphs can be used to explicitly differentiate between asserted and non-asserted RDF triples. In the example provided in Figure 5, `:s_2` can be asserted independently from `:s_1`, meaning named graphs can be used to express that “`<:e_3 tido:supports :h_2>` according to `:analyst`” without considering “`<:e_3 tido:supports :h_2>`” to be an asserted fact.

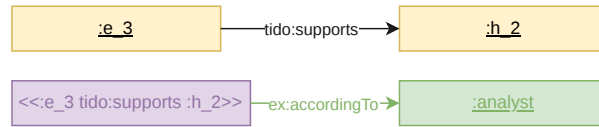
RDF-star (r-star). Similarly to singleton properties, RDF-star, originally referred to as RDF*, was also developed to address the flaws of the standard reification method, most notably the excess number of triples needed to refer to a statement and the unnecessary complexity of queries over these reified statements [20]. RDF* relies on the notion of a ‘quoted triple’, which is used to refer to the original triple. Initially, RDF* was implemented as a wrapper for standard reification; the example in Listing

Listing 5: r-star reified triple

```

1 :e_3 tido:supports :h_2 .
2 << :e_3 tido:supports :h_2 >>
  ex:accordingTo :analyst .

```

Figure 6: r-star reified graph

5 could be mapped to the example in Listing 1 through a mapping function. This means that the original triple is not implied by the reified triple. However, since standard reification uses a blank node to refer to a reified triple, the id of the node used to refer to the reified triple may be unique for every quoted instance of the original triple. This potentially unwanted behaviour is addressed in the RDF-star community report draft which states that the quoted triples are unique; whenever the quoted triple `<< :e_3 tido:supports :h_2 >>` appears, it always denotes one and the same thing [21]. This implementation resolves a common misconception on the interpretation of quoted triples, but also results in the need for more complex referential structures when referring to different assertions of identical triples.

4. Requirement Satisfaction

Requirements 1-7 identified in Section 2 consists of two groups: Requirements 1-4 are properties that can be satisfied by the reification methods, whereas requirements 5-7 are comparative; the objective is to find the *best* option. As such, satisfaction of requirements 1-4 can be determined by an analysis of the inherent properties of the reification methods, while satisfaction of requirements 5-7 needs to be determined according to a comparative study. Such comparative studies are based on an instantiated ontology or a knowledge graph. The TIDO ontology is still under development and implementation details are yet to be determined. Therefore, no instantiation resembling in-use conditions exists. Instead, the presented analysis is derived from the published literature of reification methods applied to a wide variety of knowledge graphs. These original comparisons were made with different KGs, triple stores, and SPARQL queries. Even the implementation of the reification methods differs. For example, Hernández et al. [2] include Wikidata’s statement representation [15] as a their implementation of n-ary relationships, while Iglesias-Molina et al. [5] make an explicit distinction between Wikidata’s statement representation and n-ary relations and analyse them separately. This ambiguity in terminology and testing conditions substantially complicates the comparison of experimental results. Therefore, the analysis presented in Section 4.2 focuses solely on general trends within published experimental results, and the justifications for the interpretation of the implementations of the reification methods are included in the appendix.

4.1. Property analysis

R1: Perspective plurality. If the original statement is inferred from the reified statement, the perspective of an analyst would by default be asserted in the global graph. This is likely to result in ambiguous scenarios when multiple perspectives are asserted simultaneously, violating the first requirement. For *s-reif*, *n-rel*, *n-graph* and *r-star*, the original statement is not entailed by the reified statement and therefore these methods satisfy requirement 1. As explained in Section 3, this only holds for *s-prop* if it is implemented under the less common RDF interpretation. The satisfaction of requirement 1 for *s-prop* is therefore implementation dependent.

R2: Past state preservation. Since RDF graphs are monotonic, new triples can always be added to a graph without requiring existing triples to be removed. This property extends to all five reification methods considered in this analysis: new meta-information, such as an updated confidence score or a newly attributed source, can be captured by asserting an additional reified statement alongside the existing one, rather than overwriting it. Consequently, provided that updates are consistently

Table 1

Requirement satisfaction according to inherent properties of the included reification methods. ✓ - satisfied by default, ~ - satisfaction depends on the implementation of the reification method, ✗ - satisfaction is difficult or impossible

	Standard reification (s-reif)	N-ary Re- lations (n-rel)	Singleton properties (s-prop)	Named graphs (n-graph)	RDF-star (r-star)
R1 - perspective plurality	✓	✓	~	✓	✓
R2 - state preservation	✓	✓	✓	✓	✓
R3 - merging reifications	~	~	✓	✓	✗
R4 - access restrictions	~	~	~	✓	✗

implemented as new assertions instead of destructive edits, the state of the graph at any prior point in time can be reconstructed by considering only the triples asserted up to that point. This behaviour is not a distinguishing property of any specific reification method, but rather a direct consequence of using RDF as an append-only representation. As such, all five reification methods are considered to satisfy requirement 2 by default.

R3: Merging reifications. Both s-prop and n-graph can define the merging behaviour at statement level. For s-prop, reified statements that should be merged can be given the same singleton property, while reified statements that should not be merged can be given distinct singleton properties. For n-graph, reified statements that should be merged can be given the same graph identifiers, and those that should not be merged can be given distinct graph identifiers. Therefore, both fully satisfy requirement 3. For s-reif and n-rel it depends on the implementation. It is common practice to refer to a reified statement with a blank node. These blank nodes are local identifiers scoped to a specific store or file. They are *not* persistent or portable, implying that graphs from different scopes should be assigned different identifiers and consequently reified statements originating from different contexts should not be merged. However, if a reified statement is instead assigned an IRI, the merging behaviour can be determined in a manner similar to s-prop and n-graph. Satisfaction is therefore implementation dependent. As stated in Section 3, r-star considers all triples quoted to be unique and, consequently, always merges reified statements. This behaviour could be overruled by assigning a unique IRI to every statement using a pattern similar to `<<:e_3 tido:supports :h_2>> ex:hasId :s1>`, but this would result in a r-star implementation of n-graph. This would defeat the purpose of using r-star and therefore r-star does not satisfy requirement 3.

R4: Access restrictions. Access requirements are best managed using n-graph. By linking access requirements to the identifiers of named graphs, this reification method can be used to precisely specify which statements are available to which analyst. Provided no blank nodes are used to identify graphs, statements can be assigned to multiple graphs [18], facilitating the creation of a complex hyper-graph structure capable of defining access restrictions over multiple sets of statements. Similar hypergraphs can be made using s-reif, n-rel, s-prop and r-star, but they would require cardinal assertions [22, 23] and as such, their satisfaction of requirement 4 depends on their implementation. For r-star there would be an additional challenge. Consider the following example: When r-star is used to express the confidence of the triple `<:e_3 prov:wasAttributedTo :teacher>`, the intuitive way to do so would be `<<:e_3 prov:wasAttributedTo :teacher>> ex:hasConfidence "0.9">`. However, in the TI domain, the identity of sources must often remain secret, but the confidence in the source attribution may be less restricted. In r-star, it is difficult to express this confidence without also disclosing the content of the original triple. Again, unique identifiers could be assigned to triples in the form of `<<:e_3 prov:wasAttributedTo :teacher>> ex:hasId :s1>`, but this would result in a r-star implementation of n-graph. This would defeat the purpose of using r-star and therefore r-star does not satisfy requirement 4.

Table 2

Coverage of reification methods (s-reif, n-rel, s-prop, n-graph, r-star), whether they include one or more quantitative analyses (Q), and whether these analyses relate to requirements 5, 6, and 7 (R5, R6, R7). The papers are presented according to their year of publication.

	Included reification methods					Conducted analysis			
	(s-reif)	(n-rel)	(s-prop)	(n-graph)	(r-star)	Q	R5	R6	R7
Alexiev (2012) [27]	✓	✓	✗	✓	✗	✗	✗	✗	✗
Hernández et al. (2015) [2]	✓	✓	✓	✓	✗	✓	✗	✓	✓
Fu et al. (2015) [23]	✗	✓	✓	✓	✗	✓	✗	✗	✓
Frey et al. (2019) [7]	✓	✓	✓	✓	✓	✓	✗	✓	✓
Sikos and Philp (2020) [28]	✓	✓	✓	✓	✓	✗	✗	✗	✗
Govindapillai et al. (2021) [29]	✓	✓	✓	✗	✓	✗	✗	✗	✗
Sen et al. (2021) [24]	✓	✓	✓	✓	✓	✓	✗	✓	✓
Orlandi et al. (2021) [3]	✓	✗	✓	✗	✓	✓	✗	✓	✓
Iglesias-Molina et al. (2023a) [5]	✓	✓	✗	✗	✓	✓	✓	✗	✓
Iglesias-Molina et al. (2023b) [30]	✓	✓	✗	✓	✓	✓	✗	✗	✗
Kieffer et al. (2023) [4]	✓	✗	✓	✓	✓	✓	✗	✓	✓
Fakih and Serrano-Alvarado (2024) [31]	✓	✓	✓	✓	✓	✗	✗	✗	✗
Pasqual et al. (2024) [32]	✗	✓	✓	✓	✓	✓	✗	✓	✓
Wu et al. (2024) [25]	✓	✓	✓	✓	✓	✗	✗	✗	✗
Blin et al. (2026) [33]	✓	✗	✓	✓	✓	✓	✗	✗	✗
Egami et al. (2026) [34]	✓	✗	✓	✗	✓	✓	✗	✗	✗
Massari et al. (2026) [26]	✗	✓	✓	✓	✓	✗	✗	✗	✗
Count:	14	13	14	13	14	11	1	6	8

4.2. Comparative analysis

For comparative analysis, a small systematic literature review was conducted. Initially, the papers by Hernandez et al. [2] and Orlandi et al. [3] were used as a starting point, combined with a bi-directional snowballing approach looking at the citations of these papers and for papers that cite either of these papers. To be included, a paper should contain a comparison between at least three of the selected reification methods. This method returned papers from Frey et al. [7], Sen et al. [24], Kieffer et al. [4] and Iglesias-Molina et al. [5].

Besides snowballing, the following query over was ran over Scopus: TITLE-ABS-KEY ((RDF AND reification)). Out of the 89 results, 6 were already identified during the snowballing phase and 10 new papers fitting the inclusion criteria were identified. One final paper was found during the analysis of the other papers, bringing the total to 17. Of the 17 identified articles, 6 conducted only a qualitative analysis, similar to the one described in Section 4.1, but specified a specific use-case such as embedding temporal information [25], provenance and tracking changes [26], or describing implicit and explicit entities and properties appearing in cultural heritage [27]. The other 11 papers supplemented their qualitative analysis with a quantitative analysis, generally based on experimental results. Of the 11 articles with quantitative results, 1 conducted a user study on the user-friendliness of the reification methods, relevant for evaluation requirement 5, 6 measured the storage space required to store or load the reified graphs, relevant for requirement 6, and 8 measured how the reification method influences query execution times, relevant for requirement 7. A complete overview of the included reification methods and the analysis covered in each identified article is presented in Table 2.

R5 - User-friendliness. Iglesias et al. [5] conducted a thorough analysis on the user-experience of different reification methods. They report that of the included methods, participants were the fastest to form r-star queries, while the proportion of correct queries was highest for n-rel. In general, s-reif was

Table 3

Results of the comparative analysis

	Relative ranking					
R5 - user-friendliness	<u>r-star</u>	≈	<u>n-rel</u>	>	<u>s-reif</u>	
R6 - optimisation: storage	<u>n-graph</u>	≈	<u>r-star</u>	≈	<u>n-rel</u>	≈ <u>s-reif</u> > <u>s-prop</u>
R7 - optimisation: query time	<u>n-graph</u>	≈	<u>r-star</u>	>	<u>n-rel</u>	≈ <u>s-reif</u> > <u>s-prop</u>

considered the least suitable from a user perspective. Unfortunately, only s-reif, n-rel, and r-star were included in the analysis. Consequently, it is impossible to say anything about to what extent n-graph or s-prop satisfy requirement 5.

R6 - Optimisation: storage. The reported storage results are difficult to compare directly, as studies differ in what they measure (file size [24] vs. cached graph size [3, 7, 2, 4, 32]), in scope (subsets of a single graph [32, 3] vs. entirely different KGs [24, 7]), and in triple store choice, with some using a single store [24, 3, 32] and others comparing multiple [4, 2, 7]. Since different triple stores optimise differently, a method that performs well in one store may perform poorly in another; for instance, Kieffer et al. [4] report that s-prop yields the smallest cached graph in both Jena and Oxigraph, but simultaneously has a cached graph size that is nearly two orders of magnitude larger than the second-worst method in GraphDB. Given this variability and the limited sample size, few clear trends emerge. The one consistent pattern is that s-prop tends to perform worst [2, 32, 24] or second-worst [7, 3], while every other method has been reported to be the best with respect to storage requirements at least once. We therefore consider s-reif, n-rel, n-graph, and r-star to have a similar performance with respect to requirement 6.

R7 - Optimisation: query time. Comparisons with respect to query times are affected by the same high-variability of testing conditions observed in the comparison of storage requirements. Next to varying graphs, graph sizes, and triple stores, the reported results also vary in the number of SPARQL queries, the number of distinct variable assignments within those queries, number of repetitions, and the measuring unit. Options for the latter include the rank of reification method per query [2], the average execution time [23], a boxplot of the execution times [3], or a histogram of the execution times [5, 32, 3, 4]. Despite this variability in testing conditions, some patterns can be identified. First, s-prop often appears to be the least performing method [2, 3, 4, 32] or second-to the least performing [7] with respect to query execution times. Second, the query times reported for n-graph are generally the best in three papers [2, 7, 4], which also holds for r-star [3, 5, 32]. Both s-reif and n-rel tend to have neither the best nor the worst performance. Therefore, we consider both n-graph and r-star to be the best options with respect to requirement 7, followed by n-rel and s-reif. The worst performing method is s-prop.

5. Discussion

The comparative analysis in Section 4.2 should be interpreted with an important caveat: RDF’s flexibility permits a wide variety of implementation choices for each reification method, and the studies included in the review do not always implement a given method in the same way. As a result, the reported performance figures do not measure the general reification method as such, but rather a specific implementation of it. This is most clearly illustrated by the divergent treatment of Wikidata’s statement model, which some authors classify as an n-rel implementation and others treat as a distinct method altogether (see Appendix B). Consequently, the rankings in Table 3 should be read as indicative of broad trends rather than as precise, implementation-independent measurements.

A dimension not addressed in the present analysis is how the different reification methods handle *referential opacity*. To illustrate, consider the following r-star triples $\langle \langle < :e_3 \text{ tido:supports} :h_2 \rangle \rangle \text{ ex:accordingTo} :analist \rangle$ and $\langle :h_2 \text{ owl:sameAs} :h_3 \rangle$. By default, r-star assumes referential opacity: a quoted triple denotes exactly the syntactic pattern it contains, so these triples do not entail

`< < :e_3 tido:supports :h_3 >> ex:accordingTo :analyst >`, even though `:h_2` and `:h_3` are asserted to be semantically the same. Under *referential transparency*, by contrast, the quoted triple would denote the concept it expresses rather than its syntactic form, so the entailment would hold and semantically equivalent quoted triples would become interchangeable.

Whether referential opacity is desirable depends on the use case. `s-reif` and `n-rel` both assume referential transparency by construction, since the reified statement links directly to the subject and object resources rather than to a fixed syntactic pattern. By contrast, `n-graph` was partly introduced to support referential opacity, treating a set of statements as an opaque, addressable unit. For `s-prop`, the implications remain an open question. Because the TI domain frequently needs to reconcile evolving or merged identities (e.g., when two entities are later found to refer to the same individual), the choice of opacity behaviour is a substantive design decision warranting further investigation.

A further emerging direction for future work concerns the effect of a chosen reification method on graph embedding techniques. Though outside the scope of the present analysis, three studies identified in the review specifically investigate this relationship [5, 33, 34], and as embeddings become increasingly relevant for downstream reasoning in TI applications, understanding this effect is important.

Finally, it is worth considering whether PROV's qualified properties could be adopted directly rather than introducing an additional, TIDO-specific reification mechanism. Since PROV-O already provides a vocabulary for qualifying relations such as attribution and association with agents, adopting it where applicable could reduce the need for a bespoke pattern and improve interoperability with existing provenance-aware tools. This possibility should be weighed against the requirements identified in Section 2 before a final design decision is made.

6. Conclusions

The Threat Intelligence domain requires an explicit representation of individual work processes of threat intelligence analysts as they make decisions with respect to deploying exceptional means or judgements in threat intelligence reports. The TIDO ontology [1] was developed to provide a unified and machine interpretable vocabulary capable of expressing these work processes. However, in its current form, the TIDO ontology is not capable of expressing important meta-information about assertions expressed in the TIDO vocabulary. Using a motivating example, 7 requirements were identified. Five well-known reification methods were compared against these 7 requirements: 4 by analysing the intrinsic properties of the reification methods, and 3 by means of a literature review over quantitative results reported in previously published comparisons between the included reification methods.

Combining the intrinsic property analysis summarised in Table 1 with the analysis of previously published quantitative studies summarised in Table 3, the `n-graph` reification method satisfies all MUST requirements (R1-R4) and, as well, requirements R6 and R7. Both `s-reif` and `n-rel` are also viable options, but would require non-standard implementations to satisfy R3 and R4, and tend to have longer query execution times. In theory, `s-prop` is a viable option as well, but only under RDF-interpretation, and both storage requirements and query times are generally worse compared to the other reification methods. Finally, `r-star` should be avoided as the assumption that all triples are unique makes it difficult to satisfy R3. Satisfying R4 is also challenging as the original triple is used as the quoted triple. This makes it difficult to refer to the reified triple without disclosing the content of the original triple.

Regarding optimisation for user-friendliness, as specified in R5, only a single article was found reporting on the user-friendliness on reification methods. A sample size of one is too limited to evaluate satisfaction of R5, and this study did not include `n-graph` or `s-prop`, leaving an important gap in the requirement satisfaction analysis.

Acknowledgments

This work was funded by the Nederlandse Organisatie voor Wetenschappelijk Onderzoek (NWO) under the KIC HEWSTI Project with grant no. KIC1.VE01.20.004 and the European Union's Horizon

Declaration on Generative AI

During the preparation of this work, the authors used Sonnet-5 in order to: Paraphrase and reword, Drafting content, Abstract drafting. Further, the authors used the Writefull overleaf extension in order to: Grammar and spelling check, Improve writing style. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

References

- [1] R. Roothaert, S. Schlobach, F. Massacci, L. Stork, TIDO: The Threat Intelligence Decision Ontology, in: Proceedings of the Knowledge Capture Conference 2025, ACM, Dayton OH USA, 2025, pp. 82–89. URL: <https://dl.acm.org/doi/10.1145/3731443.3771351>. doi:10.1145/3731443.3771351.
- [2] D. Hernández, A. Hogan, M. Kroetzsch, Reifying RDF: What Works Well With Wikidata?, in: Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems, Bethlehem, PA, USA, 2015.
- [3] F. Orlandi, D. Graux, D. O’Sullivan, Benchmarking RDF Metadata Representations: Reification, Singleton Property and RDF, in: 2021 IEEE 15th International Conference on Semantic Computing (ICSC), IEEE, Laguna Hills, CA, USA, 2021, pp. 233–240. URL: <https://ieeexplore.ieee.org/document/9364401/>. doi:10.1109/ICSC50631.2021.00049.
- [4] M. Kieffer, G. Fakih, P. Serrano Alvarado, Evaluating Reification with Multi-Valued Properties in a Knowledge Graph of Licensed Educational Resources, in: M. Acosta, S. Peroni, S. Vahdati, A.-L. Gentile, T. Pellegrini, J.-C. Kalo (Eds.), Studies on the Semantic Web, IOS Press, 2023. URL: <https://ebooks.iospress.nl/doi/10.3233/SSW230008>. doi:10.3233/SSW230008.
- [5] A. Iglesias-Molina, K. Ahrabian, F. Ilievski, J. Pujara, O. Corcho, Comparison of Knowledge Graph Representations for Consumer Scenarios, in: T. R. Payne, V. Presutti, G. Qi, M. Poveda-Villalón, G. Stoilos, L. Hollink, Z. Kaoudi, G. Cheng, J. Li (Eds.), The Semantic Web – ISWC 2023, volume 14265, Springer Nature Switzerland, Cham, 2023, pp. 271–289. URL: https://link.springer.com/10.1007/978-3-031-47240-4_15. doi:10.1007/978-3-031-47240-4_15, series Title: Lecture Notes in Computer Science.
- [6] Office of Civil Liberties, Privacy, and Transparency, Annual Statistical Transparency Report, Technical Report 13, 2026.
- [7] J. Frey, K. Müller, S. Hellmann, E. Rahm, M.-E. Vidal, Evaluation of metadata representations in RDF stores, Semantic Web 10 (2019) 205–229. URL: <https://journals.sagepub.com/doi/full/10.3233/SW-180307>. doi:10.3233/SW-180307.
- [8] S. Sen, M. C. Malta, D. Katoriya, B. Dutta, A. Dutta, Labeled k-partite Graph for Statement Annotation in the Web of Data, in: 2020 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT), IEEE, Melbourne, Australia, 2020, pp. 63–71. URL: <https://ieeexplore.ieee.org/document/9457722/>. doi:10.1109/WIIAT50758.2020.00014.
- [9] O. Lassila, R. R. Swick, Resource Description Framework (RDF) Model and Syntax Specification, 1999. URL: <https://www.w3.org/TR/1999/REC-rdf-syntax-19990222/>.
- [10] P. Hayes, RDF Semantics, 2004. URL: <https://www.w3.org/TR/2004/REC-rdf-mt-20040210/#Reif>.
- [11] P. J. Hayes, P. F. Patel-Schneider, RDF 1.1 Semantics, 2014. URL: <https://www.w3.org/TR/rdf11-mt/>.
- [12] P. Ostrowski, Analysis of RDF reification approaches for the European Agency for Railways Knowledge Graph, Ph.D. thesis, Universidad Politécnica de Madrid, 2024.
- [13] N. Noy, A. Rector, Defining N-ary Relations on the Semantic Web, Technical Report W3C Working Group Note 12 April 2006, 2006. URL: <https://www.w3.org/TR/swbp-n-aryRelations/>.

- [14] T. Lebo, S. Sahoo, D. McGuinness, K. Belhajjame, J. Cheney, D. Corsar, D. Garijo, S. Soiland-Reyes, S. Zednik, J. Zhao, PROV-O: The PROV Ontology, 2013. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>, (W3C Recommendation).
- [15] F. Erxleben, M. Günther, M. Krötzsch, J. Mendez, D. Vrandečić, Introducing Wikidata to the Linked Data Web, in: P. Mika, T. Tudorache, A. Bernstein, C. Welty, C. Knoblock, D. Vrandečić, P. Groth, N. Noy, K. Janowicz, C. Goble (Eds.), *The Semantic Web – ISWC 2014*, volume 8796, Springer International Publishing, Cham, 2014, pp. 50–65. URL: http://link.springer.com/10.1007/978-3-319-11964-9_4. doi:10.1007/978-3-319-11964-9_4, series Title: *Lecture Notes in Computer Science*.
- [16] V. Nguyen, O. Bodenreider, A. Sheth, Don't like RDF reification?: making statements about statements using singleton property, in: *Proceedings of the 23rd international conference on World wide web*, ACM, Seoul Korea, 2014, pp. 759–770. URL: <https://dl.acm.org/doi/10.1145/2566486.2567973>. doi:10.1145/2566486.2567973.
- [17] P. F. Patel-Schneider, Contextualization via Qualifiers, in: *Joint Proceedings of the International Workshops on Contextualized Knowledge Graphs, and Semantic Statistics co-located with 17th International Semantic Web Conference (ISWC 2018)*, Monterey, USA, 2018.
- [18] J. J. Carroll, C. Bizer, P. Hayes, P. Stickler, Named graphs, *Journal of Web Semantics* 3 (2005) 247–267. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1570826805000235>. doi:10.1016/j.websem.2005.09.001.
- [19] J. J. Carroll, C. Bizer, P. Hayes, P. Stickler, Semantic Web Publishing using Named Graphs, in: *Proceedings of the ISWC*04 Workshop on Trust, Security, and Reputation on the Semantic Web*, Hiroshima, Japan, 2004.
- [20] O. Hartig, Foundations of RDF* and SPARQL*: An alternative Approach to Statement-Level Metadata in RDF, in: *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management and the Web*, Montevideo, Uruguay, 2017. URL: <https://ceur-ws.org/Vol-1912/paper12.pdf>.
- [21] D. Arndt, J. Broekstra, B. DuCharme, O. Lassila, P. F. Patel-Schneider, E. Prud'hommeaux, T. J. Thibodeau, B. Thompson, RDF-star and SPARQL-star, Draft Community Group Report, 2026. URL: https://w3c-cg.github.io/rdf-star/cg-spec/editors_draft.html.
- [22] A. Gibson, B. Mons, Towards Computational Evaluation of Evidence for Scientific Assertions with Nanopublications and Cardinal Assertions, in: *Proceedings of the 5th International Workshop on Semantic Web Applications and Tools for Life Sciences*, Paris, France, 2012.
- [23] G. Fu, E. Bolton, N. Queralt-Rosinach, L. I. Furlong, V. Nguyen, A. Sheth, O. Bodenreider, M. Dumontier, Exposing Provenance Metadata Using Different RDF Models, in: *Proceedings of the International Conference on Semantic Web Applications and Tools for Life Sciences*, 2015. URL: http://www.swat4ls.org/wp-content/uploads/2015/10/SWAT4LS_2015_paper_11.pdf.
- [24] S. Sen, D. Katoriya, A. Dutta, B. Dutta, RDF : An alternative approach for representing, storing, and maintaining meta-knowledge in web of data, *Expert Systems with Applications* 179 (2021) 115043. URL: <https://linkinghub.elsevier.com/retrieve/pii/S095741742100484X>. doi:10.1016/j.eswa.2021.115043.
- [25] D. Wu, H.-T. Wang, A. U. Tansel, A survey for managing temporal data in RDF, *Information Systems* 122 (2024) 102368. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0306437924000267>. doi:10.1016/j.is.2024.102368.
- [26] A. Massari, S. Peroni, F. Tomasi, I. Heibi, Representing provenance and track changes of cultural heritage metadata in RDF: a survey of existing approaches, *Digital Scholarship in the Humanities* 41 (2026) i196–i212. URL: https://academic.oup.com/dsh/article/41/Supplement_1/i196/8219704. doi:10.1093/llc/fqaf076.
- [27] V. Alexiev, Types and Annotations for CIDOC CRM Properties, *Digital Presentation and Preservation of Cultural and Scientific Heritage* 2 (2012) 38–51. URL: <https://dipp.math.bas.bg/dipp/article/view/dipp.2012.2.23>. doi:10.55630/dipp.2012.2.23.
- [28] L. F. Sikos, D. Philp, Provenance-Aware Knowledge Representation: A Survey of Data Models and Contextualized Knowledge Graphs, *Data Science and Engineering* 5 (2020) 293–316. URL:

<https://link.springer.com/10.1007/s41019-020-00118-0>. doi:10.1007/s41019-020-00118-0.

- [29] S. Govindapillai, L.-K. Soon, S.-C. Haw, An empirical study on Resource Description Framework reification for trustworthiness in knowledge graphs, *F1000Research* 10 (2021) 881. URL: <https://f1000research.com/articles/10-881/v2>. doi:10.12688/f1000research.72843.2.
- [30] A. Iglesias-Molina, J. Toledo, O. Corcho, D. Chaves-Fraga, Re-Construction Impact on Metadata Representation Models, in: *Proceedings of the 12th Knowledge Capture Conference 2023*, ACM, Pensacola FL USA, 2023, pp. 197–205. URL: <https://dl.acm.org/doi/10.1145/3587259.3627554>. doi:10.1145/3587259.3627554.
- [31] G. Fakihi, P. Serrano-Alvarado, A survey on SPARQL query relaxation under the lens of RDF reification, *Semantic Web* (2024) 1–48. URL: <https://journals.sagepub.com/doi/full/10.3233/SW-243621>. doi:10.3233/SW-243621.
- [32] V. Pasqual, G. Manzano, E. Uzeir, F. Tomasi, F. Vitali, Evaluation of Expressing without asserting approaches in RDF. The case of Conjectures., in: *Proceedings of the 20th Conference on Information and Research science Connecting to Digital and Library science(formerly the Italian Research Conference on Digital Libraries)*, 2024.
- [33] I. Blin, I. Tiddi, A. Ten Teije, Measuring the Impact of Narrative Complexity on Knowledge Graph Embeddings, in: D. Garijo, S. Kirrane, A. Salatino, C. Shimizu, M. Acosta, A. G. Nuzzolese, S. Ferrada, T. Souldard, K. Kozaki, H. Takeda, A. L. Gentile (Eds.), *The Semantic Web – ISWC 2025*, volume 16140, Springer Nature Switzerland, Cham, 2026, pp. 481–501. URL: https://link.springer.com/10.1007/978-3-032-09527-5_26. doi:10.1007/978-3-032-09527-5_26, series Title: Lecture Notes in Computer Science.
- [34] S. Egami, K. Matsushita, T. Ugai, K. Fukuda, Comparison of Metadata Representation Models for Knowledge Graph Embeddings, in: B. Villazón-Terrazas, F. Ortiz-Rodriguez, S. Tiwari, T. Riechert, E. Marx (Eds.), *Knowledge Graphs and Semantic Web*, volume 16373, Springer Nature Switzerland, Cham, 2026, pp. 41–56. URL: https://link.springer.com/10.1007/978-3-032-13109-6_4. doi:10.1007/978-3-032-13109-6_4, series Title: Lecture Notes in Computer Science.
- [35] V. Nguyen, J. Leeka, O. Bodenreider, A. Sheth, A Formal Graph Model for RDF and Its Implementation, 2016. URL: <http://arxiv.org/abs/1606.00480>. doi:10.48550/arXiv.1606.00480, arXiv:1606.00480 [cs.DB].
- [36] P. Groth, A. Gibson, J. Velterop, The anatomy of a nanopublication, *Information Services and Use* 30 (2010) 51–56. URL: <https://journals.sagepub.com/doi/10.3233/ISU-2010-0613>. doi:10.3233/ISU-2010-0613.

A. Method interpretations

Notes on Table 2:

- Sen et al. [24] introduce two variants each for n-rel and s-prop. For n-rel, they cite [15] and [2], but appear to repeat the T-box instantiation for every occurrence of a property. This would explain why URIs that typically function as predicates appear at the object position of a triple, and is consistent with their reported results, in which this pattern occurs roughly twice as often as for s-prop. For example, given the triple $\langle s, p, o \rangle$, they represent it as:

1	<code>:s :p_{statement} :X .</code>
2	<code>:X :p_{value} :o .</code>
3	<code>:p_{statement} :statementProperty p .</code>
4	<code>:p_{value} :valueProperty p .</code>

While the latter two triples only need to be instantiated once, the presented results are consistent with these triples being instantiated for every occurrence of p . Therefore only the results reported under their second n-rel variant were included, which more closely matches the interpretation described in [13] and [7].

- For s-prop, Sen et al. [24] distinguish two implementations. The first follows the original formulation by Nguyen et al. [16], which, as with all other reification methods considered here, represents triples using a directed labelled graph (DLG). The second, also introduced by Nguyen et al. [35], instead uses a labelled directed multigraph with triple nodes (LDM-3N), representing each triple as three nodes connected by a pair of edges; a key advantage of this representation is a reduction in the number of disconnected subgraphs. As not all results reported in [24] distinguish between the two representations, only results reported under the DLG representation were included.
- Sen et al. [24] additionally compare graphs in which reified nodes carry associated meta-data triples against graphs in which these meta-data triples are omitted. We only include the results for graphs with meta-data triples.
- Iglesias-Molina et al. [5] extended Orlandi et al.’s [3] analysis with N-ary relationships and wikidata qualifiers.
- Fu et al. [23] include nano-publications in their comparison. As nano-publications are serialised using named graphs [36], their results are included under n-graph. They further distinguish, for both s-prop and n-rel, between implementations with and without cardinal assertions; we consider cardinal assertions to be a non-standard implementation choice and therefore only include results for the implementations without them.
- Frey et al. [7] use Blazegraph’s native “reification done right” as their r-star implementation. This was close to the inception of RDF-star, so their reported results may be affected by immature optimisation.
- Pasqual et al. [32] and Massari et al. [26] both refer to the ‘wikidata’ reification method. These results are included under the N-ary reification method.
- Alexiev [27] describes “long-cuts” (or “split properties”), which use an intermediate node to represent the reified statement. We consider this an early variant of the Wikidata qualifiers approach, and therefore classify it as a form of n-rel.